Technical Blocker Mitigation — Defender for Endpoint

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Technical Blocker Mitigation: Defender for Endpoint 5

What you get . . . 5

What this delivers . . . 5

The standard deployment model . . . 5

Boundary . . . 5

Effort . . . 6

1. Solution Overview — Technical Blocker Mitigation: Defender for Endpoint 6

What this solution is . . . 6

How Defender for Endpoint onboarding actually works (so you can find the break) . . . 6

The standard deployment model (applied to a blocker) . . . 7

Scope . . . 7

Out of scope . . . 7

Connections and authentication used . . . 7

2. Requirements Guide — Technical Blocker Mitigation: Defender for Endpoint 8

2.1 Read-only access to the Defender portal and Graph . . . 8

2.2 The stalled rollout's onboarding channel identified . . . 8

2.3 The affected device subset defined . . . 8

2.4 Outbound reachability to the Defender service endpoints (from a sample affected device) . . . 9

2.5 The local MDE diagnostic tooling available (where the fix needs it) . . . 9

2.6 A place to store the diagnostic output and support bundle . . . 9

Checklist . . . 9

Global Requirements — Defender (category-level) 9

G1. Microsoft 365 / Entra tenant . . . 10

G2. Defender licensing . . . 10

G3. Administrative roles (least privilege) . . . 10

G4. Onboarding reach to endpoints / identity / workloads . . . 10

G5. Network access to Defender service endpoints . . . 10

G6. A pilot ring . . . 10

G7. Log Analytics workspace (where required) . . . 11

G8. Response governance . . . 11

3. Implementation Guide — Technical Blocker Mitigation: Defender for Endpoint 11

Naming (set once, reuse verbatim) . . . 11

Step 1 — Assess: pinpoint the blocker (read-only) . . . 11

Step 2 — Assess: test connectivity where the chain looks broken (read-only) . . . 12

Step 3 — Configure: apply the one targeted fix (manual, documented) . . . 12

Step 4 — Pilot: re-test the fix on the affected subset only . . . 13
Step 5 — Validate: confirm the blocker is cleared . . . 13
Step 6 — Expand: resume the stalled rollout . . . 13
Step 7 — Escalate (only if it is a service issue, not your config) . . . 13
5. Testing Guide — Technical Blocker Mitigation: Defender for Endpoint 14
Test 1 — Blocker pinpointed (baseline captured) . . . 14
Test 2 — Service-endpoint connectivity . . . 14
Test 3 — Device onboarding cleared . . . 14
Test 4 — Sensor health restored . . . 14
Test 5 — Telemetry flowing . . . 14
Test 6 — Intune connector healthy (Intune channel) . . . 15
Test 7 — Before/after evidence produced . . . 15
Test 8 — Escalation bundle valid (only if escalating) . . . 15
6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Endpoint 15
Device not onboarding . . . 15
Sensor Inactive / Impaired communication . . . 15
No telemetry after onboarding . . . 16
MDE–Intune connector broken . . . 16
Licensing / activation blocker . . . 16
Script cannot authenticate . . . 16
When to escalate to Microsoft support . . . 16
7. Best Practices — Technical Blocker Mitigation: Defender for Endpoint 17
Diagnose before you change anything . . . 17
Change one thing, on the affected subset, then verify . . . 17
Keep everything read-only until you deliberately fix . . . 17
Connectivity is the usual suspect for health blockers . . . 17
Let telemetry and health warm up . . . 17
Keep licensing and preview facts current . . . 18
Escalate with evidence, not a description . . . 18
Defer the security decisions to the owners . . . 18
Where to go next . . . 18
8. Deliverables Summary — Technical Blocker Mitigation: Defender for Endpoint 18
Documentation . . . 18
PowerShell scripts (Microsoft Graph Security / advanced-hunting KQL — all read-only) . . . 19
What the buyer receives . . . 19
Outcome . . . 19

Appendix A — PowerShell Scripts 21

Executive Summary — Technical Blocker Mitigation: Defender for Endpoint

White-label · Defender

A Defender for Endpoint rollout almost always stalls on the same short list of blockers: devices that will not onboard, sensors reporting **Inactive** or **Impaired communication**, missing EDR/antivirus telemetry, or a broken **Defender for Endpoint** ↔ **Intune connector**. Left undiagnosed, a stalled rollout burns days of guesswork while the fleet stays unprotected. This solution is a **diagnostic-first playbook**: read-only discovery that pinpoints *which* blocker you actually have, the **targeted, documented manual fix** for that blocker, and the verification that proves it is cleared — so a stuck deployment gets moving again fast.

This is a **run-it-yourself** package written for an administrator with no prior deep MDE troubleshooting experience. Every script is **read-only** — it collects state and pinpoints the blocker; it never changes your configuration. Any fix is a **documented manual step** you apply deliberately, on the affected subset first.

What you get

- A plain-language **Solution Overview** of how MDE onboarding, sensor health, telemetry, and the Intune connector fit together — and where each one breaks.
- A **Requirements Guide** with verify / implement-if-missing for the diagnostic-specific prerequisites.
- An **Implementation Guide** that walks the diagnostic loop end to end — pinpoint the blocker, apply the targeted fix, re-test the affected subset, confirm cleared, resume the rollout.
- Five **read-only PowerShell scripts** (Microsoft Graph Security + advanced-hunting KQL) for discovery, service-endpoint connectivity testing, blocker validation, a status report, and a support-ready data bundle for escalation.
- **Testing** and **Troubleshooting** guides that map every common symptom to its cause and fix.

What this delivers

A previously stuck Defender for Endpoint rollout **moving again** — the blocker diagnosed, the targeted fix applied and verified, and, if the root cause is a Microsoft service issue rather than your configuration, a **support-ready diagnostic bundle** and clear escalation criteria so a support case gets resolved faster.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess with read-only discovery to pinpoint the blocker, apply the targeted fix, pilot the fix on the affected device subset, validate the blocker is cleared, then expand by resuming the stalled rollout.

Boundary

This is a **diagnostic and troubleshooting** solution. It gets a *stalled* MDE deployment unstuck; it does **not** replace the full onboarding engagement (see the adjacent *Defender XDR Deployment & Unified Onboarding* solution) or a posture-improvement engagement. All scripts are **read-only**. Response actions (device isolation, investigation-package collection) stay **approval-gated** and are the SOC's call;

incident-response and licensing determinations remain the customer's security, operations, and licensing functions — this guide provides the technical diagnosis and the documented fix, not the decision.

Effort

A competent administrator can run the discovery, identify the blocker, apply the documented fix, and verify clearance for a single blocker class in a focused session. Fleet-wide re-onboarding after the fix scales with device count.

1. Solution Overview — Technical Blocker Mitigation: Defender for Endpoint

What this solution is

This is a **diagnostic and troubleshooting playbook** for a **stalled Microsoft Defender for Endpoint (MDE) rollout**. It does not re-teach onboarding from scratch — it assumes you have started and hit a wall. Its job is to answer three questions quickly and in order:

1. **Which blocker do I actually have?** (read-only discovery, not guesswork)
2. **What is the exact, documented fix?** (a targeted manual step, applied deliberately)
3. **Is the blocker cleared?** (verification that proves the fix worked before you resume)

It targets the four blocker classes that stall almost every MDE deployment:

• **Devices not onboarding** — a device stays *"Can be onboarded"* in the portal and never becomes managed.

• **Sensor health failures** — an onboarded device reports **Inactive**, **Impaired communication**, or **No sensor data**.

• **Missing telemetry** — the device is onboarded and healthy, but EDR/antivirus events do not appear in advanced hunting or Secure Score.

• **MDE ↔ Intune connector issues** — the connector shows a disabled/expired state, so the auto-generated onboarding package never reaches devices.

How Defender for Endpoint onboarding actually works (so you can find the break)

MDE has a signal chain, and a blocker is always a break in one specific link:

1. **Licensing/activation** — the tenant has a Defender for Endpoint plan (P1/P2) and the service is activated in `security.microsoft.com`. *Verify current plan requirements in your tenant.*

2. **Onboarding channel** — Intune (default), Configuration Manager, Group Policy, or the local onboarding script pushes the onboarding configuration to the device. With Intune, the **Microsoft Defender for Endpoint connector** must be **Enabled** so the onboarding blob is generated and consumed automatically.

3. **The sensor onboards** — the device applies the onboarding configuration and registers with the service; it appears under **Assets** → **Devices** as **Onboarded**.

4. **The sensor reports** — the onboarded device needs **outbound HTTPS reachability** to the Defender service endpoints; if a proxy/firewall blocks them, the sensor onboards but then goes **Inactive / Impaired communication**.

5. **Telemetry lands** — healthy sensors stream to the advanced-hunting tables (`DeviceInfo`, `DeviceEvents`, `DeviceNetworkEvents`, etc.) and to Secure Score.

Find the earliest broken link and you have found the blocker. The discovery script reports the state of each link so you are not guessing.

The standard deployment model (applied to a blocker)

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand. Assess*** *— run read-only discovery and connectivity checks to pinpoint the earliest broken link.* ***Configure*** *— apply the single documented fix for that blocker (a manual step, not a script action).* ***Pilot*** *— re-test the fix on the* ***affected device subset*** *only.* ***Validate*** *— confirm telemetry/health is restored and the blocker is cleared.* ***Expand*** *— resume the stalled rollout across the remaining devices.*

Scope

- Diagnosing and clearing the four blocker classes above for a stalled MDE rollout.
- Read-only discovery, connectivity testing, validation, status reporting, and a support-ready bundle.
- Documented manual fixes for each blocker, applied affected-subset-first.
- Clear escalation criteria and the exact data to collect when the root cause is a Microsoft service issue.

Out of scope

- **First-time onboarding of a greenfield tenant** — that is the *Defender XDR Deployment & Unified Onboarding* solution; this one unblocks a rollout already in progress.
- **Posture/optimization work** (Secure Score improvement, ASR tuning, hardening) — see the adjacent CSPM / Secure Score and endpoint-security optimization solutions.
- **Response and incident handling** — device isolation, investigation packages, and automated response stay **approval-gated** and are the SOC's decision.
- **Licensing determinations** — the guide tells you *which* plan a feature needs and how to verify it; buying/assigning it is the customer's licensing function.

Connections and authentication used

- The **unified Defender portal** (`security.microsoft.com`) — Assets, Settings → Endpoints, device health.
- **Microsoft Graph Security** and the **Microsoft 365 Defender advanced-hunting API** (via `Microsoft.Graph` PowerShell modules) for read-only discovery, validation, reporting, and KQL hunting.

• **Intune / Microsoft Endpoint Manager** (`intune.microsoft.com`) — the Defender for Endpoint connector and EDR onboarding policy state.

• Least-privilege roles: **Security Reader** for all read-only diagnostics; **Security Administrator** only to apply a configuration fix; **Global Administrator** only for one-time first-service activation.

2. Requirements Guide — Technical Blocker Mitigation: Defender for Endpoint

Verify **every** requirement before diagnosing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access to service endpoints, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running the MDE blocker diagnostics**. Objects you create are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Read-only access to the Defender portal and Graph

Description: Security Reader (or higher) in the Defender portal and consent to the read-only Microsoft Graph scopes the diagnostic scripts use (`SecurityEvents.Read.All`, `ThreatHunting.Read.All`, `DeviceManagementManagedDevices.Read.All`, `DeviceManagementConfiguration.Read.All`, `Directory.Read.All`). **Why:** Every script in this solution is read-only; it needs read access to device health, advanced hunting, and the Intune connector/policy state to pinpoint the blocker. **Verify:** Sign in to `security.microsoft.com`; confirm **Assets** → **Devices** and **Advanced hunting** load. Run `TBMMDE-Discovery.ps1` — it reports the roles and scopes it resolved. **Implement-if-missing:** Assign **Security Reader**; grant admin consent to the read scopes above (the script requests them on first run).

2.2 The stalled rollout's onboarding channel identified

Description: Know which channel was used to onboard the affected devices — **Intune** (default), Configuration Manager, Group Policy, or the local onboarding script. **Why:** The fix for "device not onboarding" differs by channel; you cannot diagnose a connector you are not using, and you must check the right policy object. **Verify:** In `intune.microsoft.com` → **Endpoint security** → **Endpoint detection and response**, confirm whether an EDR onboarding policy exists; note the channel. `TBMMDE-Discovery.ps1` reports the Intune connector state and any EDR policy it finds. **Implement-if-missing:** Confirm the channel with whoever started the rollout; if no channel exists at all, this is a *first-time onboarding* task (adjacent solution), not a blocker.

2.3 The affected device subset defined

Description: A named subset of the devices exhibiting the blocker — the ones you will re-test the fix against before resuming the whole rollout. Name it `MDE Blocker Triage – Affected Ring – Diagnose`. **Why:** Fixes are validated on the affected subset first (per `global-requirements-defender.md` G6), so you prove the fix before touching the rest of the fleet. **Verify:** The group exists in Entra/Intune and contains the affected devices; `TBMMDE-Validate.ps1` checks membership and per-device state. **Implement-if-missing:** Create the Entra/Intune group `MDE Blocker Triage – Affected Ring – Diagnose` and add the devices that show the symptom.

2.4 Outbound reachability to the Defender service endpoints (from a sample affected device)

Description: The ability to test, from a representative affected device, outbound HTTPS to the documented Defender for Endpoint service URLs/IP ranges. **Why:** The single most common cause of **Inactive / Impaired communication** is a proxy or firewall blocking the service endpoints; you must be able to test this from where the device actually sits. **Verify:** Run `TBMMDE-Connectivity.ps1` on (or against) a sample affected device to check reachability to the documented endpoints. *Verify the current Defender endpoint URL/IP list in your tenant — Microsoft updates it.* **Implement-if-missing:** Obtain access to a sample device (or its network path) and the current allow-list; work with networking to test the documented endpoints.

2.5 The local MDE diagnostic tooling available (where the fix needs it)

Description: On Windows devices, the built-in `MpCmdRun.exe` and the Microsoft **Defender client analyzer** are available for local, on-device confirmation when a fix requires it. **Why:** Some sensor-health blockers are only confirmable on the device itself (agent version, connectivity self-test); the analyzer produces the exact bundle Microsoft support asks for. **Verify:** Confirm `%ProgramFiles%\Windows Defender\MpCmdRun.exe` exists on a sample device; know where to obtain the current client analyzer. *Verify the current analyzer download in your tenant.* **Implement-if-missing:** Note the analyzer download location for the escalation step; no install is needed for the read-only cloud diagnostics.

2.6 A place to store the diagnostic output and support bundle

Description: A secure location for the JSON snapshots the scripts write and for the support bundle `MDE Support Bundle – <case> – Escalate`. **Why:** The before/after snapshots prove the blocker was cleared, and the bundle is what you attach to a Microsoft support case. **Verify:** A working folder exists with least-privilege access; `TBMMDE-Report.ps1` writes there. **Implement-if-missing:** Create a restricted working folder; do not store tenant identifiers in shared/unsecured locations.

Checklist

- Security Reader + read-only Graph scopes consented — `TBMMDE-Discovery.ps1`
- Onboarding channel of the stalled rollout identified (Intune / CM / GPO / script)
- Affected device subset `MDE Blocker Triage – Affected Ring – Diagnose` created
- Service-endpoint reachability testable from a sample affected device — `TBMMDE-Connectivity.ps1`
- Local MDE diagnostic tooling (`MpCmdRun.exe` / client analyzer) located
- Secure working folder for snapshots and the support bundle

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Technical Blocker Mitigation: Defender for Endpoint

Follow these steps **in order**. This is a **diagnostic loop**, not a build: you assess to pinpoint the blocker, apply one targeted documented fix, re-test the affected subset, and only then resume the rollout. All scripts are **read-only**; every *fix* is a manual step you apply deliberately. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Assess (pinpoint) → Configure (fix) → Pilot (re-test affected subset) → Validate (confirm cleared) → Expand (resume rollout). Never disable an existing control to "test," and keep response actions approval-gated throughout.*

Naming (set once, reuse verbatim)

- Affected device subset (Entra/Intune): `MDE Blocker Triage – Affected Ring – Diagnose`
- Advanced-hunting saved query (onboarding/telemetry check): `MDE Onboarding Blocker Check – Tenant – Diagnose`
- Support bundle output: `MDE Support Bundle – <case> – Escalate`
- Discovery snapshot file: `MDE-blocker-discovery.json`

Step 1 — Assess: pinpoint the blocker (read-only)

1. Sign in to `security.microsoft.com` as **Security Reader**.

2. Run `TBMMDE-Discovery.ps1`. It captures the full signal chain in one snapshot (`MDE-blocker-discovery.json`): licensed Defender plans, the account's roles, the **Intune ↔ MDE connector** state, any EDR onboarding policy and its assignment, device onboarding coverage, and a per-state device count (Onboarded / Can be onboarded / Inactive / Impaired).

3. In **Entra/Intune**, create the group `MDE Blocker Triage – Affected Ring – Diagnose` and add the devices showing the symptom.

4. Read the snapshot top-down against the signal chain (Overview §"How onboarding actually works") and identify the **earliest broken link**. That link is your blocker class:

• No plan / not activated → **licensing/activation blocker**

• Connector disabled/expired, or policy unassigned → **connector / onboarding blocker**

• Devices "Can be onboarded" → **onboarding blocker**

• Devices Onboarded but Inactive/Impaired → **sensor-health / connectivity blocker**

• Healthy but no hunting rows → **telemetry blocker**

Step 2 — Assess: test connectivity where the chain looks broken (read-only)

1. If the blocker looks like sensor-health/connectivity (or you are not yet sure), run `TBMMDE-Connectivity.ps1` on or against a **sample device from** `MDE Blocker Triage – Affected Ring – Diagnose`.

2. It tests outbound HTTPS to the documented Defender service endpoints and reports each as reachable/blocked. *Verify the current endpoint URL/IP list in your tenant.*

3. Record the result — a blocked endpoint confirms a proxy/firewall blocker and points the fix at networking, not at MDE.

Step 3 — Configure: apply the one targeted fix (manual, documented)

Apply **only** the fix for the blocker you identified. Each is a deliberate manual step — the scripts never make these changes.

• **Licensing / activation:** Assign the Defender for Endpoint plan (Admin center → **Billing** → **Licenses**), then complete first-service activation in `security.microsoft.com` → **Settings** → **Endpoints** → **Onboarding**. *Verify the exact plan the feature needs in your tenant.*

• **Connector / onboarding (Intune channel):** `intune.microsoft.com` → **Endpoint security** → **Microsoft Defender for Endpoint** → set **Connection status: Enabled** (re-establish the connection if it shows disabled/expired). Then confirm the EDR policy exists, uses **client configuration package type = Auto from connector**, and is **assigned** to `MDE Blocker Triage – Affected Ring – Diagnose`. Force an Intune sync on a sample device.

• **Onboarding (GPO/script channel):** Re-apply the current onboarding package from **Settings** → **Endpoints** → **Onboarding** to the affected OU/device; confirm the onboarding scheduled task/service ran.

• **Sensor-health / connectivity:** Fix the blocked endpoints found in Step 2 at the proxy/firewall (allow the documented Defender endpoints per `global-requirements-defender.md` G5). If connectivity is fine, run the Microsoft Defender **client analyzer** on the device to confirm agent health and reconnect.

• **Telemetry:** If onboarded and healthy but no hunting rows, allow warm-up time (tables populate over hours), confirm the device is not excluded by a device group scope, and confirm advanced hunting is enabled for the tenant.

Step 4 — Pilot: re-test the fix on the affected subset only

1. Apply the fix to (or force a sync on) the devices in `MDE Blocker Triage – Affected Ring – Diagnose` only — not the whole fleet.

2. Give the change time to take effect (Intune sync / sensor check-in can take minutes to hours).

3. Move to validation before touching any other device.

Step 5 — Validate: confirm the blocker is cleared

1. Run `TBMMDE-Validate.ps1`. It re-checks the same signal chain for the affected subset and returns a **pass/fail** per link (onboarded state, sensor health, recent telemetry).

2. Run `TBMMDE-Report.ps1` to produce a before/after status report from the Step 1 snapshot versus now — this is your evidence the blocker cleared.

3. Complete the **Testing Guide** for the specific blocker class before expanding.

Step 6 — Expand: resume the stalled rollout

1. With the affected subset green, resume the rollout to the remaining devices in waves (broaden the Intune assignment / re-apply the package to the next ring).

2. Re-run `TBMMDE-Validate.ps1` after each wave and compare with `TBMMDE-Report.ps1`.

3. If a wave re-introduces the symptom, return to Step 1 for that subset — the blocker class may differ by network segment or OS.

Step 7 — Escalate (only if it is a service issue, not your config)

1. If discovery, connectivity, and the fix all check out but the blocker persists, it may be a Microsoft service issue.

2. Run `TBMMDE-SupportBundle.ps1` to assemble `MDE Support Bundle – <case> – Escalate` — the read-only diagnostic snapshot, connectivity results, affected device IDs, and timestamps — plus run the on-device **client analyzer** for the local bundle.

3. Open a Microsoft support case, attach the bundle, and reference the escalation criteria in the Troubleshooting Guide. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

Order of operations

`Assess (discovery + connectivity) → identify earliest broken link → apply the one targeted fix → re-test the affected subset → validate cleared → resume rollout → escalate with a bundle only if it is a service issue`

5. Testing Guide — Technical Blocker Mitigation: Defender for Endpoint

Run these checks after applying a fix on the affected subset (`MDE Blocker Triage – Affected Ring – Diagnose`) and again after each expansion wave. Each test has a **pass/fail gate**; do not resume the wider rollout until the affected subset passes the test for its blocker class. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Blocker pinpointed (baseline captured)

Procedure: Run `TBMMDE-Discovery.ps1` and confirm `MDE-blocker-discovery.json` was written with the licensing, connector, policy, and per-state device counts populated. **Pass:** A single earliest broken link is identifiable from the snapshot; the affected devices are in `MDE Blocker Triage – Affected Ring – Diagnose`. **Fail:** Snapshot empty or access-denied — fix read access (Requirements 2.1) and re-run.

Test 2 — Service-endpoint connectivity

Procedure: Run `TBMMDE-Connectivity.ps1` against a sample affected device. **Pass:** All documented Defender service endpoints report reachable. **Fail:** One or more endpoints blocked — see Troubleshooting *Sensor Inactive / Impaired communication*; the fix is a proxy/firewall allow, not an MDE change.

Test 3 — Device onboarding cleared

Procedure: Run `TBMMDE-Validate.ps1`. In `security.microsoft.com` → **Assets** → **Devices**, confirm each device in the affected subset now shows **Onboarded** (not "Can be onboarded"). **Pass:** Every affected device is onboarded; the script reports onboarded state. **Fail:** A device stays "Can be onboarded" — see Troubleshooting *Device not onboarding*.

Test 4 — Sensor health restored

Procedure: Run `TBMMDE-Validate.ps1`. In **Assets** → **Devices**, confirm sensor health is **Active** (not Inactive / Impaired communication / No sensor data) for the affected subset. **Pass:** Sensors are Active and reporting. **Fail:** Sensor still Inactive/Impaired — re-check Test 2 connectivity, then see Troubleshooting *Sensor Inactive / Impaired communication*.

Test 5 — Telemetry flowing

Procedure: Run `TBMMDE-Validate.ps1`, which executes the saved query `MDE Onboarding Blocker Check – Tenant – Diagnose` to count recent rows in `DeviceInfo` / `DeviceEvents` for the affected devices. **Pass:** Recent rows appear for the affected devices within the expected window. **Fail:** No rows — allow warm-up time, confirm device-group scoping, see Troubleshooting *No telemetry after onboarding*.

Test 6 — Intune connector healthy (Intune channel)

Procedure: Run `TBMMDE-Discovery.ps1` and confirm the reported **MDE ↔ Intune connector** state is **Enabled**; in Intune confirm the EDR policy is assigned to the affected subset. **Pass:** Connector Enabled and policy assigned. **Fail:** Connector disabled/expired or policy unassigned — see Troubleshooting *MDE–Intune connector broken*.

Test 7 — Before/after evidence produced

Procedure: Run `TBMMDE-Report.ps1` and confirm it emits a before/after comparison from the Step 1 snapshot versus current state. **Pass:** The report shows the blocker's link moved from broken to cleared. **Fail:** No change captured — re-run discovery to refresh the baseline.

Test 8 — Escalation bundle valid (only if escalating)

Procedure: If the blocker persists after a correct fix, run `TBMMDE-SupportBundle.ps1` and confirm `MDE Support Bundle – <case> – Escalate` contains the discovery snapshot, connectivity results, affected device IDs, and timestamps. **Pass:** A complete, attachable bundle exists (plus the on-device client-analyzer output). **Fail:** Bundle incomplete — re-run discovery/connectivity first, then rebuild.

Exit gate

The affected subset passes the tests for its blocker class **before** you resume the wider rollout (Implementation Step 6). Re-run Tests 3–5 and 7 after each expansion wave and compare against the saved baseline.

6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Endpoint

The four blocker classes in **symptom** → **cause** → **fix** form, plus escalation. Run `TBMMDE-Discovery.ps1` and `TBMMDE-Connectivity.ps1` first — most causes surface there. This *is* the troubleshooting solution, so this guide is the core reference.

Device not onboarding

Symptom: A device in `MDE Blocker Triage – Affected Ring – Diagnose` stays **"Can be onboarded"** in **Assets** → **Devices** and never becomes managed. **Cause:** The onboarding configuration never reached or applied on the device — the EDR onboarding policy is unassigned, the Intune connector is off (so no package is generated), or the GPO/script channel did not run. **Fix:** Confirm the channel (Requirements 2.2). Intune: set **Endpoint security** → **Microsoft Defender for Endpoint** → **Connection status: Enabled**, confirm the EDR policy uses **Auto from connector** and is assigned to the affected subset, then force an Intune sync. GPO/script: re-apply the current onboarding package from **Settings** → **Endpoints** → **Onboarding** and confirm the onboarding task/service ran. Re-run `TBMMDE-Validate.ps1`.

Sensor Inactive / Impaired communication

Symptom: A device shows **Onboarded** but sensor health is **Inactive**, **Impaired communication**, or **No sensor data**. **Cause:** The onboarded sensor cannot reach the Defender service endpoints — a proxy/firewall block is the most common root cause; less often an out-of-date agent or a stopped service. **Fix:** Run `TBMMDE-Connectivity.ps1` on a sample device; allow any blocked Defender endpoints at the proxy/firewall (per `global-requirements-defender.md` G5 — *verify the current endpoint list*). If connectivity is clean, run the Microsoft Defender **client analyzer** on the device to confirm agent version/health and reconnect. Re-run `TBMMDE-Validate.ps1` and confirm **Active**.

No telemetry after onboarding

Symptom: A device is Onboarded and Active, but no rows appear in `DeviceInfo` / `DeviceEvents` and it is absent from Secure Score. **Cause:** Warm-up not complete, the device is scoped out by a device group, or advanced hunting is not enabled for the tenant. **Fix:** Allow warm-up time (tables populate over hours). Confirm the device is not excluded by a device-group scope. Confirm advanced hunting is available. Run `TBMMDE-Validate.ps1`, which executes `MDE Onboarding Blocker Check – Tenant – Diagnose` to count recent rows; re-check after warm-up.

MDE–Intune connector broken

Symptom: Devices will not onboard via Intune; `TBMMDE-Discovery.ps1` reports the connector as **disabled/expired**, or onboarding packages are stale. **Cause:** The Defender for Endpoint ↔ Intune connection lapsed (token expiry, first-time link never completed, or it was toggled off). **Fix:** `intune.microsoft.com` → **Endpoint security** → **Microsoft Defender for Endpoint** → re-establish the connection so **Connection status: Enabled**; confirm the app-level connection in `security.microsoft.com` → **Settings** → **Endpoints** → **Advanced features** → **Microsoft Intune connection** is on. Re-assign the EDR policy to the affected subset and force a sync. Re-run `TBMMDE-Discovery.ps1`.

Licensing / activation blocker

Symptom: Onboarding options are missing, or the service is not activated in the portal. **Cause:** No Defender for Endpoint plan assigned, or first-service activation was never completed. **Fix:** Assign the plan (Admin center → **Billing** → **Licenses**), then complete activation in **Settings** → **Endpoints** → **Onboarding**. *Verify the exact plan the feature needs — availability changes; confirm in your tenant.* Re-run `TBMMDE-Discovery.ps1`.

Script cannot authenticate

Symptom: `TBMMDE-*.ps1` fails on connect or returns access-denied. **Cause:** The Microsoft Graph modules are missing, consent to the read scopes was not granted, or the account lacks Security Reader. **Fix:** Let the script install the modules (it checks on run); sign in with `Connect-MgGraph` and consent to the read-only scopes; confirm **Security Reader** (or higher). Re-run.

When to escalate to Microsoft support

Escalate — and attach `MDE Support Bundle – <case> – Escalate` — when **all** of the following hold:

- Discovery shows the plan assigned, service activated, connector Enabled, and policy assigned.
- Connectivity to the documented endpoints is clean from the affected device.
- The correct documented fix was applied and warm-up time elapsed, yet the device still will not onboard or stays Inactive.

In that case the root cause is likely a **service-side issue**, not your configuration. Run `TBMMDE-SupportBundle.ps1`, run the on-device **client analyzer**, open the case, and attach both. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

7. Best Practices — Technical Blocker Mitigation: Defender for Endpoint

Field-proven guidance for diagnosing a stalled MDE rollout cleanly, fixing the *right* thing, and not making it worse.

Diagnose before you change anything

- Run `TBMMDE-Discovery.ps1` **first** and read the signal chain top-down. The earliest broken link is the blocker; fixing a later link while an earlier one is broken wastes time.
- Resist the urge to "re-onboard everything." Most stalls are one broken link (a disabled connector, a blocked endpoint) affecting many devices — fix the link, not the symptoms.

Change one thing, on the affected subset, then verify

- Apply exactly **one** documented fix, to `MDE Blocker Triage – Affected Ring – Diagnose` only, then run `TBMMDE-Validate.ps1`. One change at a time keeps cause and effect clear.
- Never resume the fleet-wide rollout until the affected subset passes its test gate.

Keep everything read-only until you deliberately fix

- Every script here is read-only by design. Treat the *fix* as a separate, conscious action taken in the portal/Intune — not something a script should ever do for you.
- Never disable an existing control (AV, an ASR rule, a policy) to "test" a blocker; that trades a rollout problem for a protection gap.

Connectivity is the usual suspect for health blockers

- When sensors onboard but go **Inactive / Impaired communication**, test the service endpoints first (`TBMMDE-Connectivity.ps1`). A proxy/firewall block is the most common root cause and the fix belongs to networking. *Verify the current Defender endpoint list — it changes.*

Let telemetry and health warm up

- Onboarding state, sensor health, and advanced-hunting tables populate over minutes to hours, not seconds. Give a fix time before calling it a failure, then re-validate.

Keep licensing and preview facts current

• Defender for Endpoint features are **plan-gated** (P1/P2). Before concluding "it's broken," confirm the feature is even licensed and activated — *verify current availability and plan requirements in your tenant.*

Escalate with evidence, not a description

• If the blocker survives a correct fix, it may be a service issue. Attach `MDE Support Bundle – <case> – Escalate` and the on-device client-analyzer output to the case. A complete bundle resolves cases far faster than a written symptom.

Defer the security decisions to the owners

• Device isolation, investigation-package collection, and automated-response levels are **the SOC/security team's** decisions, kept approval-gated. This solution diagnoses and clears the *deployment* blocker; the response posture stays with the customer.

Where to go next

Once the rollout is unblocked, natural next steps in the catalog:

• **Defender XDR Deployment & Unified Onboarding** to finish onboarding the remaining pillars and correlate incidents in one portal.

• **Microsoft Defender for Cloud — CSPM & Secure Score Remediation** to move from "onboarded" to "hardened."

• The endpoint-security **optimization** solutions (ASR, attack-surface reduction, EDR tuning) to raise protection once telemetry is flowing.

• The **Sentinel** SOC/XDR integration to stream MDE signal into a SIEM for long-term hunting.

8. Deliverables Summary — Technical Blocker Mitigation: Defender for Endpoint

Documentation

Document	Purpose
Executive Summary	The blockers, the diagnostic-first approach, the outcome, the boundary, effort
1. Solution Overview	The four MDE blocker classes, the onboarding signal chain, where each link breaks
2. Requirements Guide	Diagnostic-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	The diagnostic loop — assess, fix, re-test the affected subset, validate, resume, escalate
5. Testing Guide	Eight pass/fail tests, one per link in the signal chain
6. Troubleshooting Guide	Every blocker class in symptom → cause → fix, plus escalation criteria
7. Best Practices	Diagnose-first discipline and where to go next

PowerShell scripts (Microsoft Graph Security / advanced-hunting KQL — all read-only)

Script	What it does
`TBMMDE-Discovery.ps1`	Read-only signal-chain snapshot — licensed plans, admin roles, MDE↔Intune connector state, EDR policy/assignment, device onboarding coverage, and per-state device counts (Onboarded / Can be onboarded / Inactive / Impaired)
`TBMMDE-Connectivity.ps1`	Read-only reachability test from a sample affected device to the documented Defender service endpoints — pinpoints proxy/firewall blocks
`TBMMDE-Validate.ps1`	Read-only pass/fail per signal-chain link for the affected subset — onboarded state, sensor health, and recent telemetry via the saved hunting query
`TBMMDE-Report.ps1`	Read-only before/after status report from the discovery baseline versus current state — the evidence the blocker cleared
`TBMMDE-SupportBundle.ps1`	Read-only assembly of the support-ready bundle (discovery snapshot, connectivity results, affected device IDs, timestamps) for a Microsoft support case

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **read-only PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable diagnostic process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pinpoint the blocker, apply one documented fix, re-test the affected subset, confirm cleared, resume the rollout.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A previously stuck Defender for Endpoint rollout moving again — the blocker diagnosed against the signal chain, the targeted documented fix applied and verified on the affected subset, and a support-ready bundle ready if the root cause turns out to be a Microsoft service issue.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TBMMDE-Connectivity.ps1

Purpose: The most common cause of a sensor that onboards but reports Inactive / Impaired communication is a proxy or firewall blocking the Defender service endpoints. This script tests outbound HTTPS reachability from the machine it runs on (a sample device from 'MDE Blocker Triage - Affected Ring - Diagnose') to a set of documented Defender endpoints and reports each as reachable or blocked. Makes NO changes. HOW TO RUN Run locally on a sample affected Windows device: powershell -ExecutionPolicy Bypass -File .\TBMMDE-Connectivity.ps1 Optionally pass your tenant's current endpoint list: .\TBMMDE-Connectivity.ps1 -EndpointFile .\mde-endpoints.txt EXPECTED RESULTS A pass/fail line per endpoint and an overall verdict. Any blocked endpoint points the fix at the proxy/firewall (allow the documented Defender endpoints), NOT at MDE configuration. .NOTES White-label. Read-only — performs connectivity probes only; changes nothing. The built-in list is a STARTING POINT — verify the current Defender endpoint URL/IP list for your tenant and geography (it changes) and pass it with -EndpointFile. > [CmdletBinding()] param([string]$EndpointFile, [string]$OutputPath = ".\MDE-connectivity.json", [int]$TimeoutSeconds = 10)

How to run: Run locally on a sample affected Windows device: powershell -ExecutionPolicy Bypass -File .\TBMMDE-Connectivity.ps1 Optionally pass your tenant's current endpoint list: .\TBMMDE-Connectivity.ps1 -EndpointFile .\mde-endpoints.txt EXPECTED RESULTS A pass/fail line per endpoint and an overall verdict. Any blocked endpoint points the fix at the proxy/firewall (allow the documented Defender endpoints), NOT at MDE configuration. .NOTES White-label. Read-only — performs connectivity probes only; changes nothing. The built-in list is a STARTING POINT — verify the current Defender endpoint URL/IP list for your tenant and geography (it changes) and pass it with -EndpointFile. > [CmdletBinding()] param([string]$EndpointFile, [string]$OutputPath = ".\MDE-connectivity.json", [int]$TimeoutSeconds = 10)

TBMMDE-Discovery.ps1

Purpose: Read-only discovery for the "Technical Blocker Mitigation — Defender for Endpoint" solution. Captures the full MDE onboarding signal chain in one snapshot so you can find the earliest broken link: licensed Defender plans, the signed-in account's roles, the MDE<->Intune connector state, any EDR onboarding policy and its assignment, device onboarding coverage, and a per-state device count. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDE-Discovery.ps1 Requires the Microsoft.Graph modules (Authentication, DeviceManagement, Identity.DirectoryManagement). Connects read-only with Connect-MgGraph and prompts for consent to read scopes only. EXPECTED RESULTS A JSON snapshot (MDE-blocker-discovery.json) plus a console summary. Read it top-down against the signal chain in the Solution Overview; the earliest broken link is the blocker. .NOTES White-label. Read-only. Reports on configuration/state only; changes nothing. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDE-blocker-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDE-Discovery.ps1 Requires the Microsoft.Graph modules (Authentication, DeviceManagement, Identity.DirectoryManagement). Connects read-only with Connect-MgGraph and prompts for consent to read scopes only. EXPECTED RESULTS A JSON snapshot (MDE-blocker-discovery.json) plus a console summary. Read it top-down against the

signal chain in the Solution Overview; the earliest broken link is the blocker. .NOTES White-label. Read-only. Reports on configuration/state only; changes nothing. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDE-blocker-discovery.json")

TBMMDE-Report.ps1

Purpose: Read-only reporting for the "Technical Blocker Mitigation — Defender for Endpoint" solution. Compares the discovery baseline (MDE-blocker-discovery.json from TBMMDE-Discovery.ps1) with a fresh live read of the same signal-chain links, and prints a before/after summary — the evidence the blocker moved from broken to cleared. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDE-Report.ps1 -Baseline .\MDE-blocker-discovery.json Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A table of each link's baseline vs current state and an overall CLEARED / STILL-BLOCKED verdict, written to MDE-blocker-report.json. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDE-blocker-discovery.json", [string]$OutputPath = ".\MDE-blocker-report.json", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDE-Report.ps1 -Baseline .\MDE-blocker-discovery.json Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS A table of each link's baseline vs current state and an overall CLEARED / STILL-BLOCKED verdict, written to MDE-blocker-report.json. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDE-blocker-discovery.json", [string]$OutputPath = ".\MDE-blocker-report.json", [int]$LookbackHours = 24)

TBMMDE-SupportBundle.ps1

Purpose: Read-only packaging for the "Technical Blocker Mitigation — Defender for Endpoint" solution. When discovery, connectivity, and the documented fix all check out but the blocker persists (likely a Microsoft service issue), this assembles the bundle 'MDE Support Bundle - <case> - Escalate': the discovery snapshot, the connectivity results, the affected device IDs/names, tenant context, and timestamps — everything to attach to a support case. Makes NO changes to the tenant. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDE-SupportBundle.ps1 -Case "MSCASE-12345" Point it at the artifacts you already produced: .\TBMMDE-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDE-blocker-discovery.json -Connectivity .\MDE-connectivity.json Requires Microsoft.Graph.Authentication for the tenant/context stamp only (read scopes). EXPECTED RESULTS A single folder 'MDE-Support-Bundle-<case>' containing a manifest and copies of the read-only artifacts. ALSO run the Microsoft Defender client analyzer ON the affected device and add its output ZIP to the folder — that is the on-device half Microsoft support asks for. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains tenant/device identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$Discovery = ".\MDE-blocker-discovery.json", [string]$Connectivity = ".\MDE-connectivity.json", [string]$Report = ".\MDE-blocker-report.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDE-SupportBundle.ps1 -Case "MSCASE-12345" Point it at the artifacts you already produced: .\TBMMDE-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDE-blocker-discovery.json -Connectivity .\MDE-connectivity.json Requires Microsoft.Graph.Authentication for the tenant/context stamp only (read scopes). EXPECTED

RESULTS A single folder 'MDE-Support-Bundle-<case>' containing a manifest and copies of the read-only artifacts. ALSO run the Microsoft Defender client analyzer ON the affected device and add its output ZIP to the folder — that is the on-device half Microsoft support asks for. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains tenant/device identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$Discovery = ".\MDE-blocker-discovery.json", [string]$Connectivity = ".\MDE-connectivity.json", [string]$Report = ".\MDE-blocker-report.json")

TBMMDE-Validate.ps1

Purpose: Read-only validation for the "Technical Blocker Mitigation — Defender for Endpoint" solution. Runs the saved advanced-hunting logic 'MDE Onboarding Blocker Check - Tenant - Diagnose' via Microsoft Graph Security to confirm, for the affected devices, the state of each signal-chain link: onboarded/sensor-health status and recent telemetry rows. Returns a pass/fail per link. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDE-Validate.ps1 Optionally scope by device name pattern: .\TBMMDE-Validate.ps1 -DeviceNameLike "PILOT-" -LookbackHours 24 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for onboarded/health and telemetry. FAIL points you to the matching Troubleshooting section for that blocker class. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$DeviceNameLike = "", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDE-Validate.ps1 Optionally scope by device name pattern: .\TBMMDE-Validate.ps1 -DeviceNameLike "PILOT-" -LookbackHours 24 Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for onboarded/health and telemetry. FAIL points you to the matching Troubleshooting section for that blocker class. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$DeviceNameLike = "", [int]$LookbackHours = 24)